ЛАБОРАТОРНА РОБОТА №6

Тема: Розробка клієнтських додатків з використанням Blazor WebAssembly

Мета: отримати навички створення та інтеграції клієнтського додатку з RESTful Web API за допомогою Blazor WebAssembly.

Хід роботи:

Завдання 1.

Створіть у своєму рішенні проект Blazor WebAssembly, який взаємодіятиме з WebAPI.

Лістинг Program.cs:

```
using Microsoft.AspNetCore.Components.Web;
using Microsoft.AspNetCore.Components.WebAssembly.Hosting;
using Microsoft.AspNetCore.Components.Authorization;
using BlazorClient;
using BlazorClient.Auth;

var builder = WebAssemblyHostBuilder.CreateDefault(args);
builder.RootComponents.Add<App>("#app");
builder.RootComponents.Add<HeadOutlet>("head::after");
builder.Services.AddScoped(sp => new HttpClient { BaseAddress = new
Uri("http://localhost:5257/") });

builder.Services.AddAuthorizationCore();
builder.Services.AddScoped<AuthenticationStateProvider, CustomAuthStateProvider>();

await builder.Build().RunAsync();
```

Розробіть сторінки та необхідні компоненти для виконання CRUD-операцій з існуючими сутностями. Додайте валідацію для створених форм.

Лістинг Vacancies.razor:

```
@page "/vacancies"
@using BlazorClient.Models
@using Microsoft.AspNetCore.Authorization
@inject HttpClient Http
@attribute [Authorize]

<h3>Вакансії</h3>

@if (!string.IsNullOrEmpty(errorMessage))
{
    <div class="alert alert-danger mb-3">@errorMessage</div>
}
```

```
@if (vacancies == null && string.IsNullOrEmpty(errorMessage))
{
    <p><em>Завантаження...</em></p>
}
else if (vacancies != null)
{
    <table class="table">
        <thead>
            <tr>
                <th>ID</th>
                <th>Назва</th>
                <th>Опис</th>
                <AuthorizeView Roles="Admin">
                    <Authorized Context="authContext">
                        <th>Дії</th>
                    </Authorized>
                </AuthorizeView>
            </tr>
        </thead>
        <tbody>
            @foreach (var vac in vacancies)
            {
                <tr>
                    <td>@vac.Id</td>
                    <td>@vac.Title</td>
                    <td>@vac.Description</td>
                    <AuthorizeView Roles="Admin">
                        <Authorized Context="authContext">
                            <td>
                                <button class="btn btn-danger btn-sm" @onclick="() =>
DeleteVacancy(vac.Id)">Видалити</button>
                            </td>
                        </Authorized>
                    </AuthorizeView>
                </tr>
            }
        </tbody>
    </table>

    <AuthorizeView Roles="Admin">
        <Authorized Context="authContext">
            <hr />
            <h4>Додати вакансію</h4>
            <EditForm Model="newVacancy" OnValidSubmit="AddVacancy">
                <DataAnnotationsValidator />
                <div class="mb-2">
                    <InputText class="form-control" placeholder="Назва" @bind-
Value="newVacancy.Title" />
            <ValidationMessage For="@(() => newVacancy.Title)" class="text-danger" />
                </div>
                <div class="mb-2">
                    <InputText class="form-control" placeholder="Опис" @bind-
Value="newVacancy.Description" />
            <ValidationMessage For="@(() => newVacancy.Description)" class="text-danger"
/>
                </div>
                <button type="submit" class="btn btn-primary">Зберегти</button>
            </EditForm>
        </Authorized>
    </AuthorizeView>
}

@code {
    private List<Vacancy>? vacancies;
    private Vacancy newVacancy = new();
```

```
    private string errorMessage = string.Empty;

    protected override async Task OnInitializedAsync() => await LoadData();

    private async Task LoadData()
    {
        try
        {
            errorMessage = string.Empty;
            vacancies = await Http.GetFromJsonAsync<List<Vacancy>>("api/vacancyapi");
        }
        catch (Exception ex) { errorMessage = $"Помилка: {ex.Message}"; }
    }

    private async Task AddVacancy()
    {
        var res = await Http.PostAsJsonAsync("api/vacancyapi", newVacancy);
        if (res.IsSuccessStatusCode)
        {
            newVacancy = new Vacancy();
            await LoadData();
        }
    }

    private async Task DeleteVacancy(int id)
    {
        var res = await Http.DeleteAsync($"api/vacancyapi/{id}");
        if (res.IsSuccessStatusCode) await LoadData();
    }
}
```

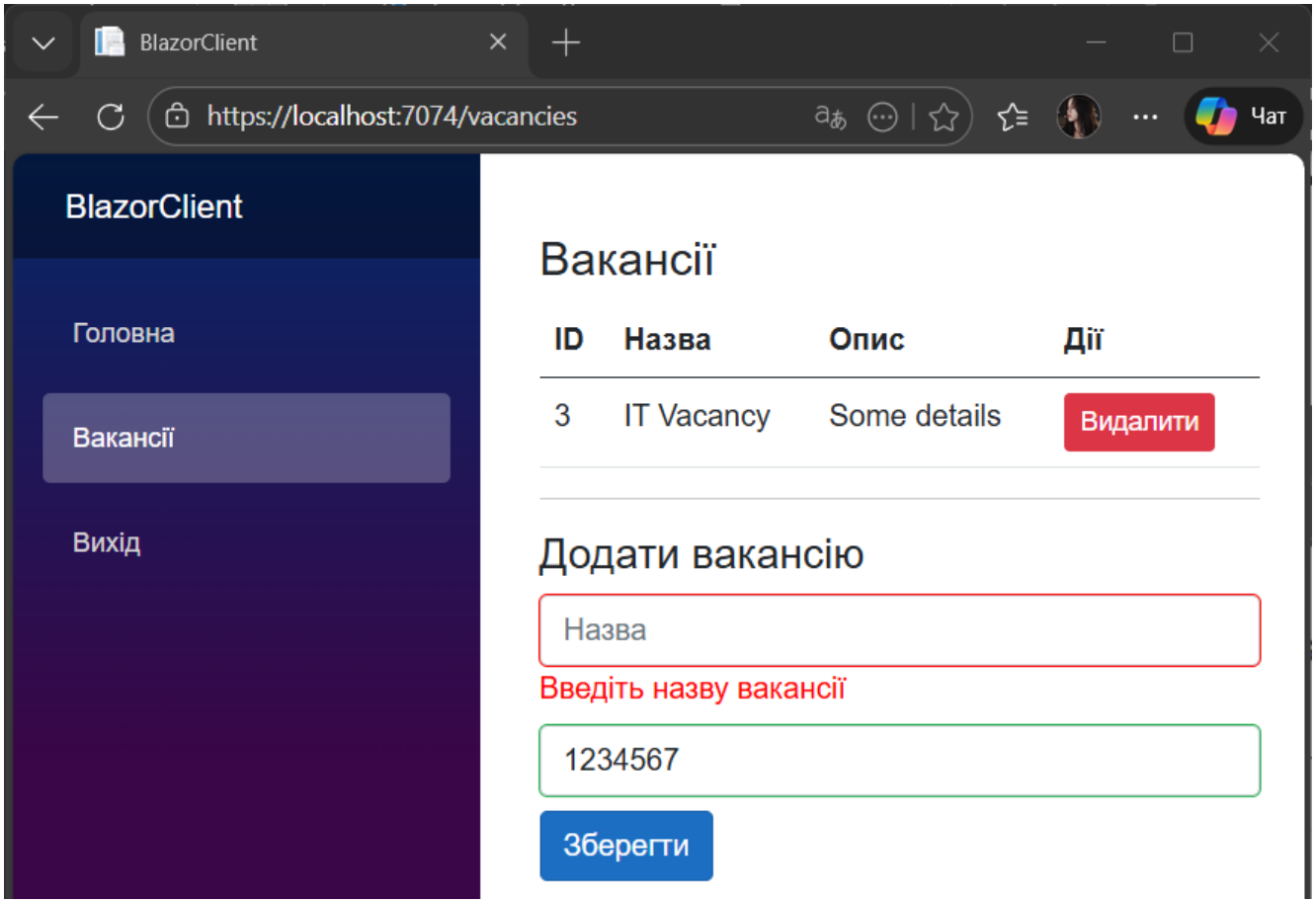

Рис 1. Сторінка з Вакансіями (вид адміністратора).

Завдання 2.

Додайте автентифікацію та авторизацію до клієнта. Додайте сторінки та відповідні форми для реєстрації та входу. Реалізуйте автентифікацію на основі JWT.

Лістинг (частковий) CustomAuthStateProvider.cs:

```
using Microsoft.AspNetCore.Components.Authorization;
using Microsoft.JSInterop;
using System.Net.Http.Headers;
using System.Security.Claims;
using System.Text.Json;

namespace BlazorClient.Auth
{
    public class CustomAuthStateProvider : AuthenticationStateProvider
    {
        private readonly IJSRuntime _jsRuntime;
        private readonly HttpClient _httpClient;
        public CustomAuthStateProvider(IJSRuntime jsRuntime, HttpClient httpClient)
        {
            _jsRuntime = jsRuntime;
            _httpClient = httpClient;
        }

        public override async Task<AuthenticationState> GetAuthenticationStateAsync()
        {
    var token = await _jsRuntime.InvokeAsync<string>("localStorage.getItem", "jwtToken");

            if (string.IsNullOrWhiteSpace(token))
            {
              return new AuthenticationState(new ClaimsPrincipal(new ClaimsIdentity()));
            }

_httpClient.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer",
token);

            var claims = ParseClaimsFromJwt(token);
            var identity = new ClaimsIdentity(claims, "jwt", "name", "role");
            var user = new ClaimsPrincipal(identity);

            return new AuthenticationState(user);
        }

        public void MarkUserAsAuthenticated(string token)
        {
_httpClient.DefaultRequestHeaders.Authorization = new AuthenticationHeaderValue("Bearer",
token);
var authenticatedUser = new ClaimsPrincipal(new ClaimsIdentity(ParseClaimsFromJwt(token),
"jwt", "name", "role"));
            var authState = Task.FromResult(new AuthenticationState(authenticatedUser));
            NotifyAuthenticationStateChanged(authState);
        }

        public void MarkUserAsLoggedOut()
        {
            _httpClient.DefaultRequestHeaders.Authorization = null;
            var anonymousUser = new ClaimsPrincipal(new ClaimsIdentity());
            var authState = Task.FromResult(new AuthenticationState(anonymousUser));
            NotifyAuthenticationStateChanged(authState);
              }
```

Лістинг (частковий) Login.razor:

```
< EditForm Model = "loginModel" OnValidSubmit = "HandleLogin" >
    < DataAnnotationsValidator />
    < InputText @bind - Value = "loginModel.Username" placeholder = "Логін" />
< InputText type = "password"@bind-Value = "loginModel.Password" placeholder = "Пароль"/>
    < button type = "submit" class= "btn btn-primary" > Увійти </ button >
</ EditForm >

@code {
    private LoginModel loginModel = new();

private async Task HandleLogin()
{
    var response = await Http.PostAsJsonAsync("api/auth/login", loginModel);

    if (response.IsSuccessStatusCode)
    {
        var result = await response.Content.ReadFromJsonAsync<AuthResponse>();

     await JSRuntime.InvokeVoidAsync("localStorage.setItem", "jwtToken", result!.Token);

((CustomAuthStateProvider)AuthStateProvider).MarkUserAsAuthenticated(result.Token);
        Navigation.NavigateTo("/");
    }
}
      }
```

Лістинг (частковий) Register.razor:

```
< EditForm Model = "newUser" OnValidSubmit = "HandleRegister" >
    < DataAnnotationsValidator />
    < InputText @bind - Value = "newUser.Username" placeholder = "Логін" />
    < InputText type = "email" @bind - Value = "newUser.Email" placeholder = "Email" />
< InputText type = "password" @bind - Value = "newUser.Password" placeholder = "Пароль"/>


    < InputSelect @bind - Value = "newUser.Role" >
        < option value = "User" > Звичайний користувач </ option >
        < option value = "Admin" > Адміністратор </ option >
    </ InputSelect >


<button type = "submit" class= "btn btn-success" > Зареєструватися </button> </EditForm >

@code {
    private User newUser = new();

private async Task HandleRegister()
{
    var response = await Http.PostAsJsonAsync("api/Users", newUser);

    if (response.IsSuccessStatusCode)
    {
        Navigation.NavigateTo("/login");
    }
}
      }
```

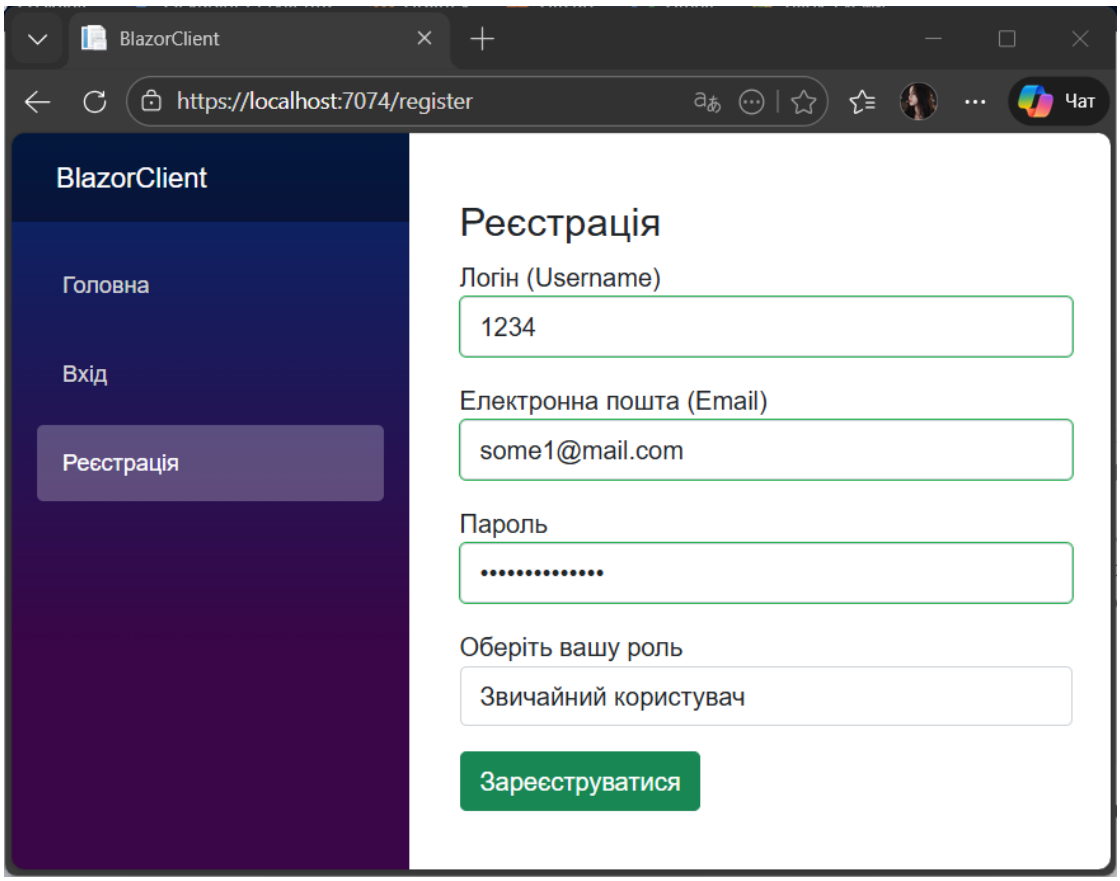

Рис. 2. Сторінка реєстрації.

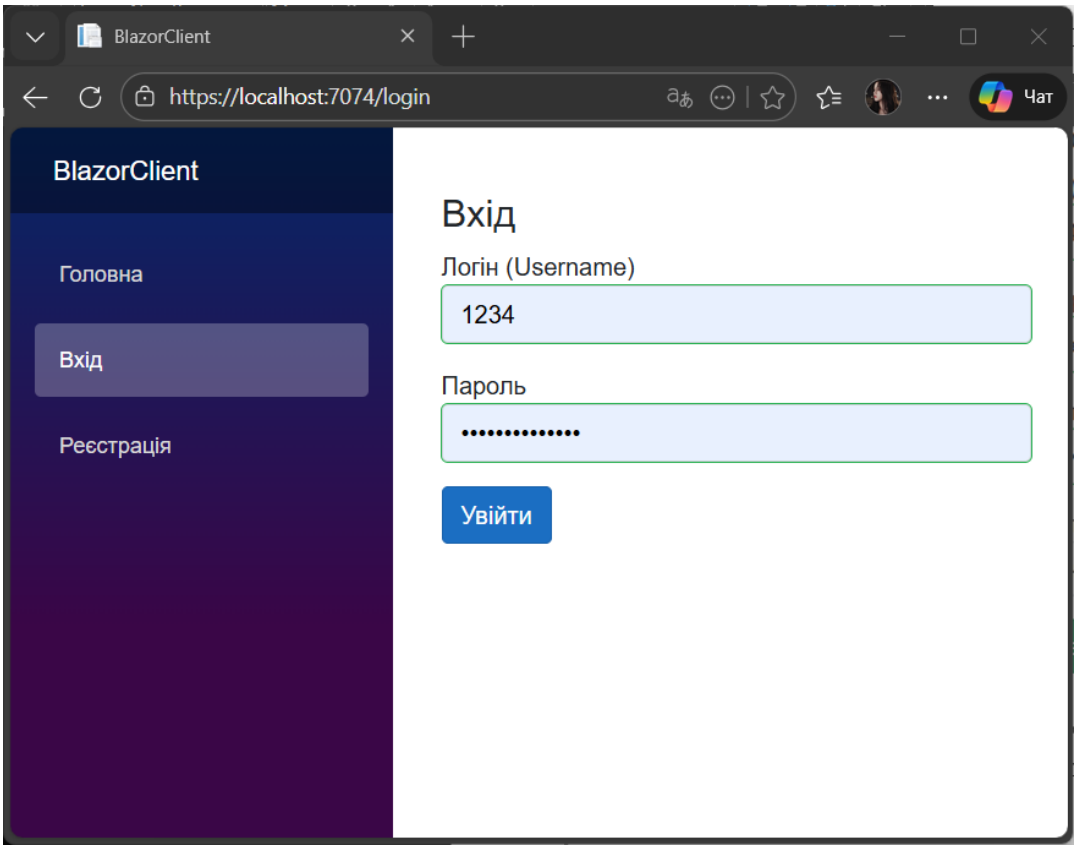

Рис. 3. Автоматичний перехід на сторінку логіну.

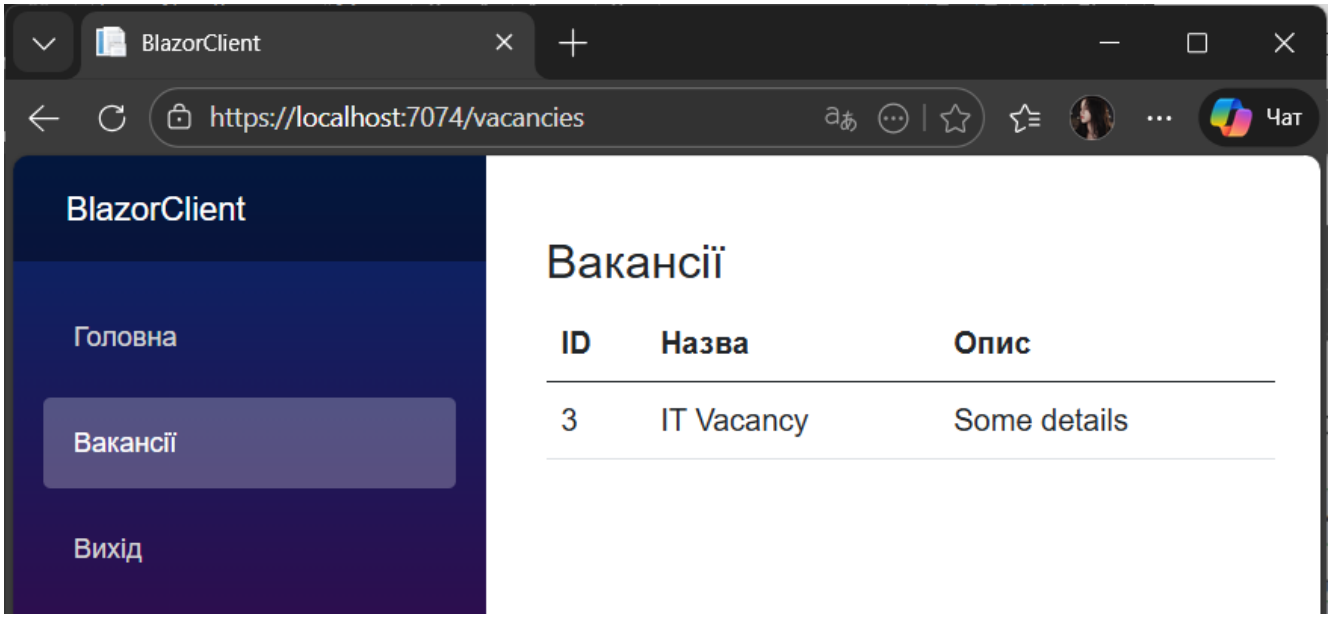

Рис. 4. Сторінка Вакансій (вид звичайного користувача)

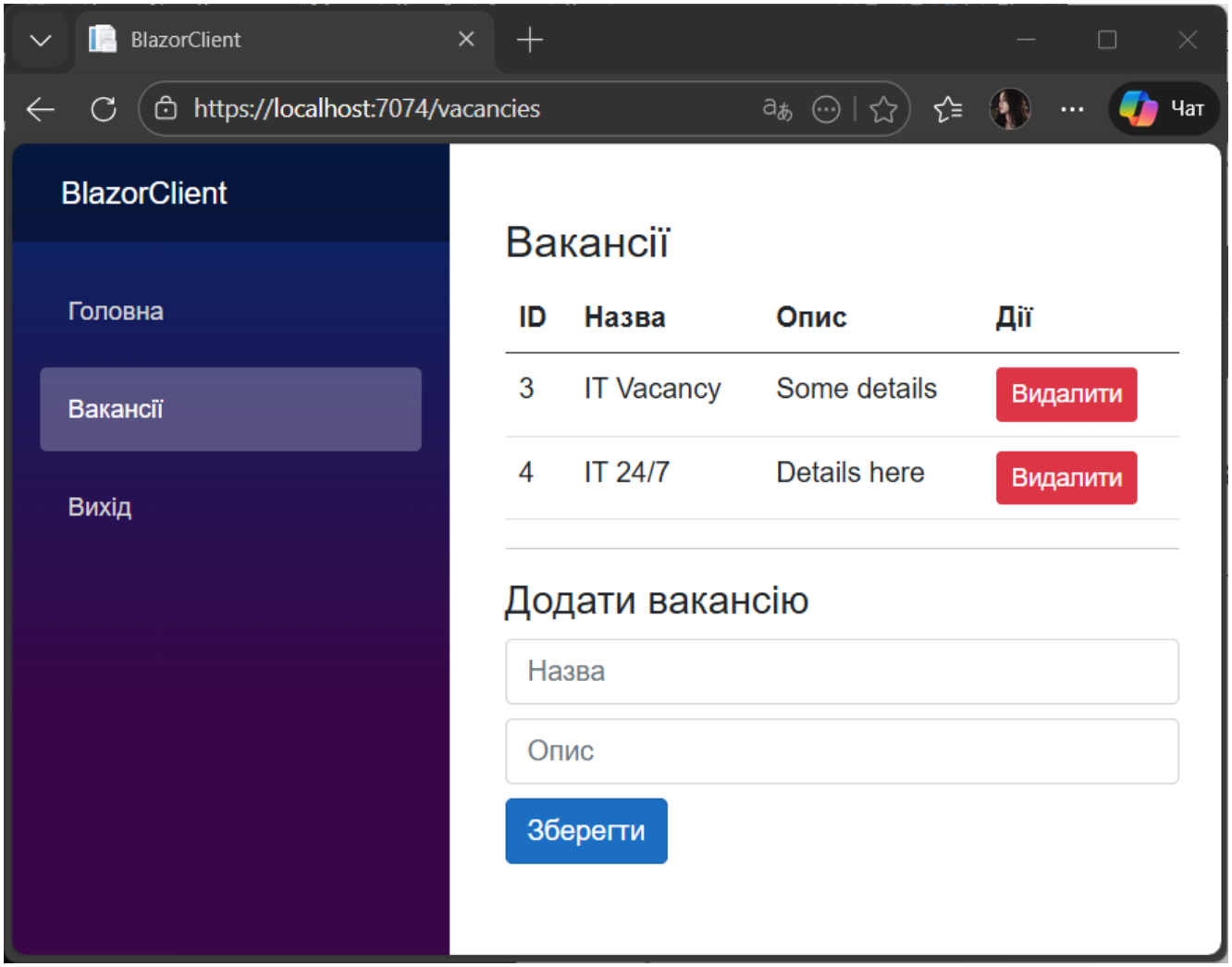

Рис. 5. Сторінка Вакансій та результат додавання вакансії (вид адміна)

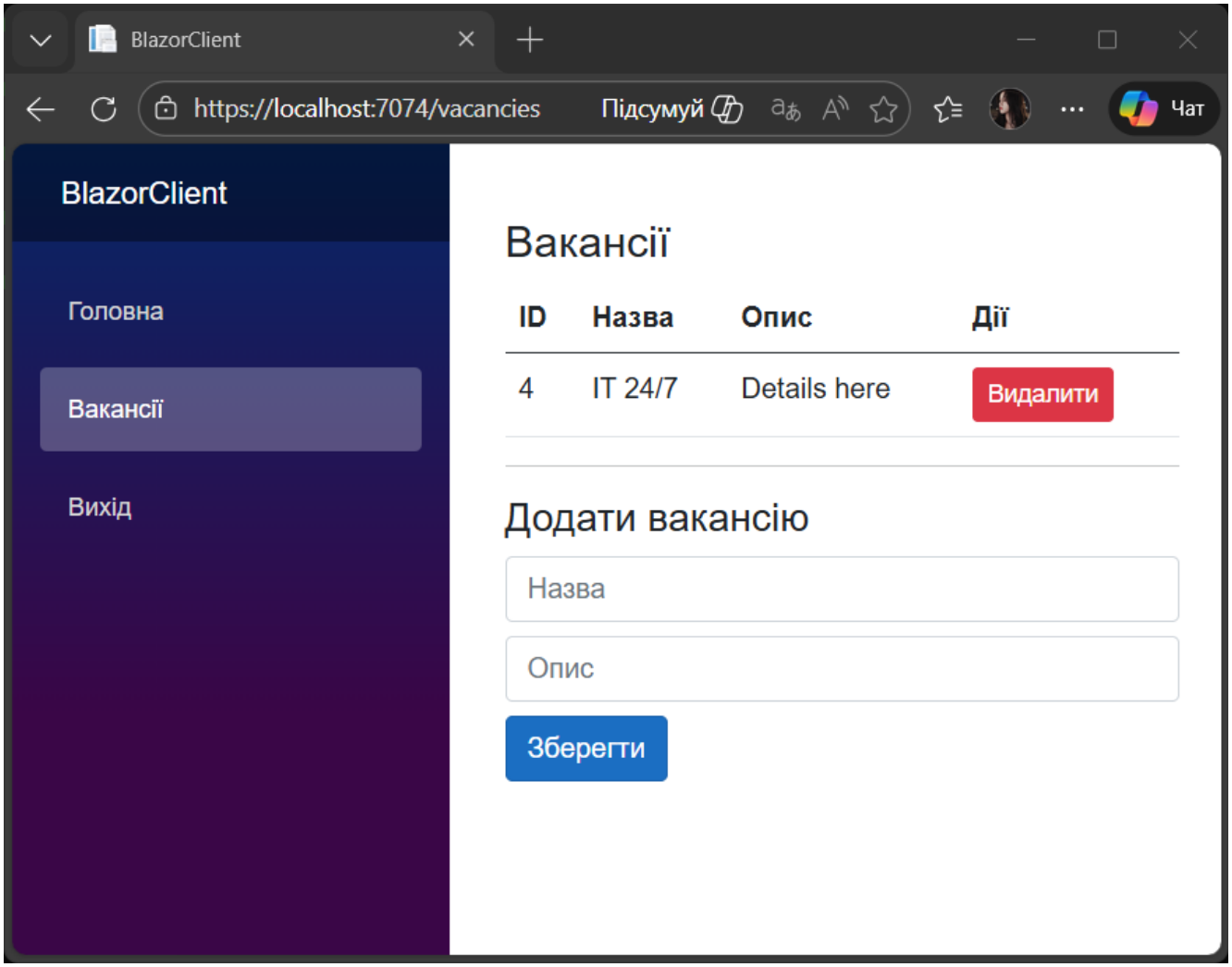

Рис. 6. Результат видалення вакансії (вид адміна)

Висновок: В ході лабораторної роботи №6 я отриматла навички створення та інтеграції клієнтського додатку з RESTful Web API за допомогою Blazor WebAssembly.

Посилання на репозиторій:
https://github.com/thlinksi/AspDotNetLabs/tree/main/Lab6